Check your app before release

Scan your website, API or code. Get one report showing what needs attention and what could not be tested.

1. What does the app do?

What you scan	What it checks
Website / UI The pages people use.	Security headers, cookies, login forms, secrets in page code, exposed files and other common concerns.
API The endpoints your app calls.	Authentication, CORS, error messages and endpoint coverage. Optional active tests include cross-user access, malformed inputs and rate limiting.
Code repository Your source-code folder.	Known dependency vulnerabilities, leaked secrets and insecure code patterns using Trivy, Gitleaks and Semgrep.

> **Only test apps you own or have permission to test.** A clean report is not proof that your application is secure. This is an early-release tool, not a full penetration test.

2. Install once

Install Python 3.10 or newer. Download the project and open a terminal in its `securerelease-cli` folder.

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e .
securerelease doctor
```

Windows PowerShell

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -e .
securerelease doctor
```

`doctor` shows installed scanner engines. Basic UI and API scans work without optional engines. Repository scans require Trivy, Gitleaks and Semgrep installed separately.

For JavaScript-rendered websites, install the browser engine:

```
python -m pip install -e ".[ui]"
playwright install chromium
```

For YAML API specifications: `python -m pip install -e ".[api]"`. JSON specifications need no extra package. Initial installation and some external scanners may need internet access.

SET UP, THEN RUN

Start your first scan

3. Easiest option: save your settings

From the project folder, run:

```
securerelease init
```

Answer the questions about your website, API and test credentials. Press Enter to skip anything you do not need. This creates a private `.env` settings file.

Or copy the example yourself: `cp .env.example .env` (PowerShell: `Copy-Item .env.example .env`). Edit the following lines:

```
SECURERELEASE_UI_URL=https://your-staging.example
SECURERELEASE_API_URL=https://api.your-staging.example
SECURERELEASE_OPENAPI_SPEC=./openapi.json
SECURERELEASE_API_TOKEN=your-test-user-token
SECURERELEASE_AUTHORIZED=false
SECURERELEASE_UI_ENGINE=crawler
```

Replace the example addresses and token. Leave unused endpoints and credentials empty. Your OpenAPI file describes your API endpoints; ask your API developer for it.

```
securerelease run
```

The app scans the targets you configured and writes `reports/securerelease-report.html`. To scan only one part: `securerelease run --profile ui` or `securerelease run --profile api`.

> **Keep credentials private.** Do not share or commit `.env`. Use test accounts. Tokens are hidden in the setup prompt, but the file stores them in plain text. Only run from a folder whose settings file you trust.

4. Alternative: use terminal commands

These do not read `.env` unless you add `--env-file .env`.

Website only

```
securerelease ui-scan --url https://your-staging.example \
  --ui-engine crawler --format html --output reports/ui.html
```

API only

```
securerelease api-scan --api-url https://api.your-staging.example \
  --openapi-spec ./openapi.json --format html --output reports/api.html
```

Commands above use macOS/Linux line continuations. In PowerShell, put each command on one line.

For authenticated APIs, set `SECURERELEASE_API_TOKEN` in `.env`, then run `securerelease run --profile api`. Terminal-only users can pass `--api-token-env API_TOKEN_A` after setting that environment variable securely.

For localhost, add `--allow-private-targets` or set `SECURERELEASE_ALLOW_PRIVATE_TARGETS=true`. Active tests need `--authorized` or `SECURERELEASE_AUTHORIZED=true`; use these only on approved test environments. Leave `SECURERELEASE_ALLOW_MUTATING=false` to avoid API writes.

UNDERSTAND YOUR RESULTS

Read and share the report

5. Open the HTML report

Double-click `reports/securerelease-report.html` to open it in a browser. On macOS you can run:

```
open reports/securerelease-report.html
```

For terminal commands that use another output name, open that file instead.

1. **Check coverage first.** Search for `UI-COVERAGE` and `API-COVERAGE`. These say how many pages or operations were reached and what was skipped. A small scan cannot assess your whole app.
2. **Read the banner and counts.** They summarize high-impact findings and execution gaps. The HTML banner uses CRITICAL/HIGH findings; a custom terminal failure threshold may differ.
3. **Use the FAIL and ERROR filters.** Review CRITICAL/HIGH findings first. ERROR means the scan needs fixing, not necessarily that the target is vulnerable.
4. **Expand a finding.** Read its asset (where), summary (what happened), evidence (what was observed) and remediation (what to do next).
5. **Fix, verify and scan again.** Confirm suspected issues with your developer. Keep both reports to compare the results.

Status	Plain-language meaning
FAIL	The check detected a problem. Review its severity and evidence.
WARN	A concern needs review; it may depend on your application's design.
ERROR	The scanner failed. Resolve the execution problem and try again.
NOT_TESTED	There is no result for this check. It is **not a pass**.
PASS	This specific check passed. Other security issues may still exist.

Example: "Password in GET request" means a login form puts the password in a URL. Ask your developer to use POST over HTTPS, then scan again.

6. Save a scan report as PDF

Open the HTML report, clear any filters, and expand the findings you want to include. Select **Print** (Ctrl+P or Command+P), choose **Save as PDF**, and save.

PDF is a static copy: filters and expandable details will not be interactive. Keep the HTML file too. The CLI currently produces HTML, not a direct `--format pdf` export.

7. If something goes wrong

Timeout: check that the URL works on your network. **Authentication rejected:** replace the expired or incorrect token. **Missing scanner:** run `securerelease doctor`. **Certificate error:** use an approved CA bundle; see the README troubleshooting section.

Exit codes: **0** = no blocking result; **1** = blocking findings (default CRITICAL/HIGH); **2** = execution error or certain required coverage gaps. `--no-fail-on-findings` ignores findings for the exit code, but execution errors still fail.

Try the local demo on macOS/Linux: `./scripts/demo.sh`. It generates `reports/demo-report.html`. Its vulnerable fixtures are for practice only; never deploy them.